

遮蔽版习题册

多邻国 Android 面试 · 17 道查错 + 9 道设计

本册只有题面，没有答案。

查错题和设计题在正式书里与答案同页，顺着读会让人产生“我看得懂”的错觉——但那是**再认**，面试考的是**回忆**。请先在这里写出你的评审，再回正式书对答案；两者的差集就是你真正要补的东西。

建议：每题限时 8-10 分钟，出声说，或者写成英文 review comment。

目录

怎么用这本册子	3
I 查错演练：协程、Flow 与并发	4
0.1 在 GlobalScope 里发起网络请求	5
0.2 在主线程上做 Room 查询与磁盘 IO	6
0.3 不可取消的轮询循环	7
0.4 async 的异常击穿了父作用域	8
0.5 在 Fragment 里收集 Flow 却没有绑定生命周期	8
0.6 被多个协程并发修改的共享可变状态	9
II 查错演练：生命周期、泄漏与列表	12
0.7 Fragment 的 binding 没有在 onDestroyView 置空	13
0.8 observe 用了 this 而不是 viewLifecycleOwner	14
0.9 ViewModel 持有 Activity Context	15
0.10 ViewHolder 在 bind 里起协程却不在回收时取消	16
0.11 每次数据变化都 notifyDataSetChanged, 且 stable ID 不稳定	17
0.12 注册了监听器却没有解注册	19
III 查错演练：Compose 与 Kotlin 惯用性	21
0.13 把 MutableStateFlow 暴露出去, 并用 SharedFlow 存持久状态	22
0.14 LaunchedEffect 的 key 用错, 副作用写在 composable 函数体里	23
0.15 collectAsState 而非 collectAsStateWithLifecycle, 以及不稳定参数引发的过度重组	24
0.16 !! 强制解包、Java 平台类型与未初始化的 lateinit	25
0.17 一整段 Java 味的 Kotlin	26
IV 设计演练：核心设计题	28
0.18 离线课程下载与进度同步	29
0.18.1 题面与常见变体	29
0.19 连胜 (Streak) 客户端	29

0.19.1 题面与常见变体	29
0.20 排行榜客户端与 XP 预测	30
0.20.1 题面与常见变体	30
0.21 客户端 A/B 实验 SDK	30
0.21.1 题面与常见变体	30
V 设计演练：进阶设计题	31
0.22 Server-Driven UI 客户端解析与渲染框架	32
0.22.1 题面与常见变体	32
0.23 图片与音频资产的缓存系统	32
0.23.1 题面与常见变体	32
0.24 推送通知的客户端接收、调度与落地	33
0.24.1 题面与常见变体	33
0.25 间隔重复练习的客户端存储与调度	33
0.25.1 题面与常见变体	33
0.26 把一个 DuoState 式全局状态重构成 MVVM	33
0.26.1 题面与常见变体	34

怎么用这本册子

查错题 (17 道): 读代码, 写出你会提的每一条意见。按优先级排序, 每条必须包含三样东西——**影响** (用户会怎样)、**触发条件** (什么情况下发生)、**具体改法** (一句话说清楚怎么改)。只写“这里有问题”不算数。

设计题 (9 道): 限时 10 分钟口述并录音。顺序是: 澄清问题 → 模块图 → 数据与存储 → 离线与同步 → 失败与回滚 → 测试与灰度。

评分方式: 对完答案后, 把你**没想到的**每一条抄进一张错题表, 并标注它属于哪一类——是不认得这个 API, 还是没有养成那个提问习惯。第二类才是需要专门练的。

一点说明: 题面保留了正式书里的场景描述, 个别题会顺带点出改动涉及的 API。这不算剧透——你要写的是一份**完整且排好序的评审**, 不是猜一个关键词。知道“这题和协程作用域有关”, 离说清楚影响、触发条件和改法还差得远。

【要点】一个自测门槛

面试前, 随机抽 5 道查错题, 每道 8 分钟。如果平均能报出**最高优先级的那一条**, 并且给得出具体改法, 这一轮基本稳了。报不出来的题, 回去看正式书对应章节的“根因分析”。

Part I

查错演练：协程、Flow 与并发

0.1 在 GlobalScope 里发起网络请求

查错 #1 | 类别: 协程 Scope

场景：用户完成一节课后，客户端要上报 session 结果、刷新 XP 和连胜，再把结果展示在完成页。[\[面经报告\]](#) 候选人报告过类似模式：代码里用 GlobalScope 发起网络请求，然后在响应后写回 UI 状态。

```
1 data class LessonResultUi(  
2     val xp: Int,  
3     val streakDays: Int,  
4     val isSynced: Boolean  
5 )  
6  
7 class LessonViewModel(  
8     private val sessionRepository: SessionRepository,  
9     private val streakRepository: StreakRepository  
10 ) : ViewModel() {  
11  
12     private val _result = MutableStateFlow<LessonResultUi?>(null)  
13     val result: StateFlow<LessonResultUi?> = _result  
14  
15     fun onLessonFinished(sessionId: String, answers: List<Answer>) {  
16         _result.value = LessonResultUi(xp = 0, streakDays = 0, isSynced = false)  
17  
18         GlobalScope.launch(Dispatchers.IO) {  
19             val submission = sessionRepository.submitSession(sessionId, answers)  
20             val streak = streakRepository.refreshStreak()  
21  
22             _result.value = LessonResultUi(  
23                 xp = submission.xpEarned,  
24                 streakDays = streak.days,  
25                 isSynced = true  
26             )  
27         }  
28     }  
29 }
```

写下你的评审。按优先级排序，每条给出：影响 / 触发条件 / 具体改法。写完再翻正式书第五章对答案。

0.2 在主线程上做 Room 查询与磁盘 IO

查错 #2 | 类别: **Dispatcher**

场景：用户打开排行榜页时，产品希望先展示本地缓存，再异步刷新远端排名。为了让首屏快，开发者写了一个“同步读缓存”的路径。

```
1 data class LeaderboardUi(  
2     val rows: List<LeaderboardRow>,  
3     val fromCache: Boolean  
4 )  
5  
6 class LeaderboardViewModel(  
7     private val repository: LeaderboardRepository  
8 ) : ViewModel() {  
9  
10     private val _ui = MutableStateFlow(LeaderboardUi(emptyList(), fromCache =  
11         true))  
12     val ui: StateFlow<LeaderboardUi> = _ui  
13  
14     fun load(courseId: String) {  
15         viewModelScope.launch(Dispatchers.Main) {  
16             val cached = repository.loadCachedLeaderboard(courseId)  
17             _ui.value = LeaderboardUi(cached, fromCache = true)  
18  
19             val fresh = repository.refreshLeaderboard(courseId)  
20             _ui.value = LeaderboardUi(fresh, fromCache = false)  
21         }  
22     }  
23  
24     class LeaderboardRepository(  
25         private val dao: LeaderboardDao,  
26         private val api: LeaderboardApi  
27     ) {  
28         suspend fun loadCachedLeaderboard(courseId: String): List<LeaderboardRow> {  
29             return dao.selectRowsForCourse(courseId)  
30         }  
31  
32         suspend fun refreshLeaderboard(courseId: String): List<LeaderboardRow> {  
33             val rows = api.fetchLeaderboard(courseId)  
34             dao.replaceRows(courseId, rows)  
35             return rows  
36         }  
37     }  
}
```

写下你的评审。按优先级排序，每条给出：**影响** / **触发条件** / **具体改法**。写完再翻正式书第五章对答案。

0.3 不可取消的轮询循环

查错 #3 | 类别：**协作式取消**

场景：排行榜页打开后每 5 秒刷新一次，用户可以一边做题一边看排名变化。开发者为了快速实现，在 ViewModel 里写了一个轮询 job。

```

1  class LeaderboardViewModel(
2      private val repository: LeaderboardRepository
3  ) : ViewModel() {
4
5      private val _rows = MutableStateFlow<List<LeaderboardRow>>(emptyList())
6      val rows: StateFlow<List<LeaderboardRow>> = _rows
7
8      private var pollingJob: Job? = null
9
10     fun startPolling(leagueId: String) {
11         pollingJob?.cancel()
12         pollingJob = viewModelScope.launch(Dispatchers.IO) {
13             while (true) {
14                 try {
15                     val latest = repository.fetchLeagueRows(leagueId)
16                     _rows.value = latest
17                     Thread.sleep(5_000)
18                 } catch (e: Exception) {
19                     logger.warn("leaderboard polling failed", e)
20                 }
21             }
22         }
23     }
24
25     fun stopPolling() {
26         pollingJob?.cancel()
27     }
28 }

```

写下你的评审。按优先级排序，每条给出：**影响** / **触发条件** / **具体改法**。写完再翻正式书第五章对答案。

0.4 async 的异常击穿了父作用域

查错 #4 | 类别: **结构化并发**

场景: 课程加载页希望并行拉取课程内容、用户进度和发音音频清单。内容是必需的；音频清单可以稍后补；进度失败时可以用本地默认值。

```

1 data class LessonBundle(
2     val lesson: LessonContent,
3     val progress: UserProgress,
4     val audio: List<AudioClip>
5 )
6
7 class LessonLoader(
8     private val lessonRepository: LessonRepository,
9     private val progressRepository: ProgressRepository,
10    private val audioRepository: AudioRepository
11 ) {
12     suspend fun load(lessonId: String): LessonBundle = coroutineScope {
13         val lesson = async { lessonRepository.fetchLesson(lessonId) }
14         val progress = async { progressRepository.fetchProgress(lessonId) }
15         val audio = async { audioRepository.prefetchAudioList(lessonId) }
16
17         LessonBundle(
18             lesson = lesson.await(),
19             progress = progress.await(),
20             audio = audio.await()
21         )
22     }
23 }

```

写下你的评审。按优先级排序，每条给出：**影响 / 触发条件 / 具体改法**。写完再翻正式书第五章对答案。

0.5 在 Fragment 里收集 Flow 却没有绑定生命周期

查错 #5 | 类别: **Flow 收集**

场景: 课程列表页订阅 ViewModel 的 StateFlow，渲染课程卡片、下载状态和继续学习按钮。
[面经报告] 面经报告过类似 Fragment 中启动协程处理网络响应、但没有正确绑定生命周期的代码。

```

1 class CourseListFragment : Fragment(R.layout.fragment_course_list) {
2

```

```

3  private val viewModel: CourseListViewModel by viewModels()
4  private var binding: FragmentCourseListBinding? = null
5  private val adapter = CourseAdapter()
6
7  override fun onCreateView(
8      inflater: LayoutInflater,
9      container: ViewGroup?,
10     savedInstanceState: Bundle?
11 ): View {
12     binding = FragmentCourseListBinding.inflate(inflater, container, false)
13     return binding!!.root
14 }
15
16 override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
17     binding!!.courseList.adapter = adapter
18
19     lifecycleScope.launch {
20         viewModel.state.collect { state ->
21             binding!!.progress.isVisible = state.loading
22             adapter.submitList(state.courses)
23             if (state.navigateToLessonId != null) {
24                 findNavController().navigate(
25                     CourseListFragmentDirections.openLesson(state.
navigateToLessonId)
26                 )
27             }
28         }
29     }
30 }
31
32 override fun onDestroyView() {
33     binding = null
34     super.onDestroyView()
35 }
36 }

```

写下你的评审。按优先级排序，每条给出：影响 / 触发条件 / 具体改法。写完再翻正式书第五章对答案。

0.6 被多个协程并发修改的共享可变状态

查错 #6 | 类别：竞态与可见性

场景：一个用户状态对象同时被 XP 上报、连胜同步和课程进度更新读写。[\[官方\]¹](https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html) 这正是官方 Android Reboot 故事里 DuoState 反模式的缩影：多个产品域共享巨大可变状态，最终带来稳定性和性能代价。

```

1  data class UserState(
2      val xp: Int,
3      val streakDays: Int,
4      val completedLessons: Set<String>
5  )
6
7  class UserStateStore(
8      private val api: UserApi,
9      private val dao: UserStateDao
10 ) {
11     private val _state = MutableStateFlow(
12         UserState(xp = 0, streakDays = 0, completedLessons = emptySet())
13     )
14     val state: StateFlow<UserState> = _state
15
16     suspend fun applyLessonResult(result: LessonResult) = coroutineScope {
17         launch(Dispatchers.IO) {
18             api.reportXp(result.sessionId, result.xpEarned)
19             _state.value = _state.value.copy(
20                 xp = _state.value.xp + result.xpEarned
21             )
22         }
23
24         launch(Dispatchers.Default) {
25             _state.value = _state.value.copy(
26                 completedLessons = _state.value.completedLessons + result.
27                 lessonId
28             )
29         }
30
31         launch(Dispatchers.IO) {
32             val streak = api.refreshStreak()
33             _state.value = _state.value.copy(streakDays = streak.days)
34             dao.save(_state.value)
35         }
36     }

```

¹<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>

写下你的评审。按优先级排序，每条给出：**影响 / 触发条件 / 具体改法**。写完再翻正式书第五章对答案。

Part II

查错演练：生命周期、泄漏与列表

0.7 Fragment 的 binding 没有在 onDestroyView 置空

查错 #7 | 类别: 内存泄漏

场景

课程详情页 LessonDetailFragment 展示课程标题、技能树入口、单词复习提醒和一个底部开始按钮。这个页面常被压入回退栈：用户从课程列表点进详情，返回列表，再点另一个课程。看起来只是一个普通 ViewBinding 写法，但如果 Fragment 实例活着而它的 View 已经销毁，旧的整棵 View 树就会被 binding 悄悄拴住。

```

1  class LessonDetailFragment : Fragment(R.layout.fragment_lesson_detail) {
2      private var _binding: FragmentLessonDetailBinding? = null
3      private val binding: FragmentLessonDetailBinding
4          get() = requireNotNull(_binding)
5
6      private val viewModel: LessonDetailViewModel by viewModels()
7      private val args: LessonDetailFragmentArgs by navArgs()
8
9      override fun onCreateView(view: View, savedInstanceState: Bundle?) {
10         super.onCreateView(view, savedInstanceState)
11         _binding = FragmentLessonDetailBinding.bind(view)
12
13         binding.toolbar.setNavigationOnClickListener {
14             findNavController().navigateUp()
15         }
16         binding.startLessonButton.setOnClickListener {
17             viewModel.startLesson(args.lessonId)
18         }
19
20         viewModel.lesson.observe(viewLifecycleOwner) { lesson ->
21             binding.title.text = lesson.title
22             binding.description.text = lesson.description
23             binding.xpReward.text = getString(R.string.xp_reward, lesson.xp)
24             binding.reviewBadge.isVisible = lesson.hasDueReview
25         }
26     }
27
28     override fun onDestroyView() {
29         super.onDestroyView()
30         binding.startLessonButton.setOnClickListener(null)
31         // BUG: _binding 仍然引用已经销毁的 View tree
32     }
33 }

```

写下你的评审。按优先级排序，每条给出：**影响** / **触发条件** / **具体改法**。写完再翻正式书第六章对答案。

0.8 observe 用了 this 而不是 viewLifecycleOwner

查错 #8 | 类别: **重复订阅**

场景

连胜卡片 `StreakCardView` 在课程详情页顶部展示用户今天是否已经完成练习。产品上它很显眼：用户从课程列表进入详情，返回，再进入另一个课程，如果连胜状态更新时卡片闪两次、弹两次动效，用户会以为奖励系统出了问题。这个 bug 的代码在首次进入时完全正常，因此特别像真实面试题。

```

1  class StreakDetailFragment : Fragment(R.layout.fragment_streak_detail) {
2      private var _binding: FragmentStreakDetailBinding? = null
3      private val binding get() = requireNotNull(_binding)
4      private val viewModel: StreakViewModel by viewModels()
5
6      override fun onCreateView(view: View, savedInstanceState: Bundle?) {
7          super.onCreateView(view, savedInstanceState)
8          _binding = FragmentStreakDetailBinding.bind(view)
9
10         viewModel.streak.observe(this) { streak ->
11             binding.streakCard.setDayCount(streak.days)
12             binding.streakCard.setFrozen(streak.isFrozen)
13             if (streak.extendedToday) {
14                 binding.streakCard.playCelebration()
15             }
16         }
17
18         viewModel.loading.observe(this) { loading ->
19             binding.progressBar.isVisible = loading
20             binding.practiceButton.isEnabled = !loading
21         }
22
23         binding.practiceButton.setOnClickListener {
24             viewModel.practiceToday()
25         }
26     }
27
28     override fun onDestroyView() {

```

```

29     _binding = null
30     super.onDestroyView()
31 }
32 }

```

写下你的评审。按优先级排序，每条给出：**影响** / **触发条件** / **具体改法**。写完再翻正式书第六章对答案。

0.9 ViewModel 持有 Activity Context

查错 #9 | 类别：**泄漏与可测试性**

场景

个人主页展示本周 XP、连续学习天数和下一枚成就徽章。ProfileViewModel 为了拼出“本周获得 120 XP”这类文案，构造函数里直接拿了 Activity 的 Context。这在功能上很顺手，却把 ViewModel 的长生命周期和 Activity 的短生命周期绑到了一起。

```

1 class ProfileViewModel(
2     private val context: Context,
3     private val userRepository: UserRepository
4 ) : ViewModel() {
5     private val _profile = MutableLiveData<ProfileUiModel>()
6     val profile: LiveData<ProfileUiModel> = _profile
7
8     fun loadProfile(userId: String) {
9         viewModelScope.launch {
10             val user = userRepository.getUser(userId)
11             val weeklyXp = userRepository.getWeeklyXp(userId)
12             val nextBadge = userRepository.getNextBadge(userId)
13
14             _profile.value = ProfileUiModel(
15                 name = user.displayName,
16                 avatarUrl = user.avatarUrl,
17                 weeklyXpText = context.getString(R.string.weekly_xp_format,
18                     weeklyXp),
19                 streakText = context.getString(R.string.streak_days_format, user
20                     .streakDays),
21                 badgeText = context.getString(R.string.next_badge_format,
22                     nextBadge.title)
23             )
24         }
25     }
26 }

```



```

22     }
23
24     override fun onCleared() {
25         super.onCleared()
26         userRepository.cancelUserRequests()
27     }
28 }

```

写下你的评审。按优先级排序，每条给出：**影响** / **触发条件** / **具体改法**。写完再翻正式书第六章对答案。

0.10 ViewHolder 在 bind 里起协程却不在回收时取消

查错 #10 | 类别：**列表与协程**

场景

排行榜每行展示头像、昵称、本周 XP 和升降名次。为了让列表先出现文字，再异步补头像与 XP，LeaderboardViewHolder.bind 里直接起协程加载。快速滚动时，RecyclerView 会复用 ViewHolder；如果旧请求回来后写进新位置，用户就会看到头像错位、XP 串行，像列表里出现了“鬼影”。

```

1  class LeaderboardAdapter(
2      private val repository: LeaderboardRepository,
3      private val scope: CoroutineScope
4  ) : RecyclerView.Adapter<LeaderboardViewHolder>() {
5      private val players = mutableListOf<LeaderboardRow>()
6
7      override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
8      LeaderboardViewHolder {
9          val binding = RowLeaderboardBinding.inflate(LayoutInflater.from(parent.
10         context), parent, false)
11         return LeaderboardViewHolder(binding, repository, scope)
12     }
13
14     override fun onBindViewHolder(holder: LeaderboardViewHolder, position: Int)
15     {
16         holder.bind(players[position])
17     }
18
19     override fun getItemCount(): Int = players.size
20 }

```

```

18
19 class LeaderboardViewHolder(
20     private val binding: RowLeaderboardBinding,
21     private val repository: LeaderboardRepository,
22     private val scope: CoroutineScope
23 ) : RecyclerView.ViewHolder(binding.root) {
24
25     fun bind(row: LeaderboardRow) {
26         binding.rank.text = row.rank.toString()
27         binding.name.text = row.displayName
28         binding.avatar.setImageResource(R.drawable.avatar_placeholder)
29         binding.weeklyXp.text = binding.root.context.getString(R.string.
loading_xp)
30
31         scope.launch {
32             val avatar = repository.loadAvatar(row.userId)
33             val xp = repository.loadWeeklyXp(row.userId)
34             binding.avatar.setImageBitmap(avatar)
35             binding.weeklyXp.text = binding.root.context.getString(R.string.
xp_amount, xp)
36         }
37     }
38 }

```

写下你的评审。按优先级排序，每条给出：**影响** / **触发条件** / **具体改法**。写完再翻正式书第六章对答案。

0.11 每次数据变化都 notifyDataSetChanged, 且 stable ID 不稳定

查错 #11 | 类别: **列表性能**

场景

排行榜是 Duolingo 产品里极适合查错的列表：实时刷新、排名变化、用户头像、徽章、滚动位置、动画全都在同一屏。社区里明确有人报告过类似真题：“Review this code that handles list updates in a RecyclerView —any performance or crash risks you’d improve?” [\[面经报告\]](#)。下面这段代码正是那类题的核心：看似只是刷新列表，实则同时破坏性能、动画和身份语义。

```

1 class LeaderboardAdapter : RecyclerView.Adapter<LeaderboardAdapter.RowViewHolder
>() {
2     private val rows = mutableListOf<LeaderboardRow>()
3

```

```

4      init {
5          setHasStableIds(true)
6      }
7
8      fun updateRows(newRows: List<LeaderboardRow>) {
9          rows.clear()
10         rows.addAll(newRows)
11         notifyDataSetChanged()
12     }
13
14     override fun getItemId(position: Int): Long {
15         return position.toLong()
16     }
17
18     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
19     ViewHolder {
20         val binding = RowLeaderboardBinding.inflate(LayoutInflater.from(parent.
21         context), parent, false)
22         return ViewHolder(binding)
23     }
24
25     override fun onBindViewHolder(holder: ViewHolder, position: Int) {
26         holder.bind(rows[position])
27     }
28
29     override fun getItemCount(): Int = rows.size
30
31     class ViewHolder(private val binding: RowLeaderboardBinding) :
32     RecyclerView.ViewHolder(binding.root) {
33         fun bind(row: LeaderboardRow) {
34             binding.rank.text = row.rank.toString()
35             binding.name.text = row.displayName
36             binding.weeklyXp.text = binding.root.context.getString(R.string.
37             xp_amount, row.weeklyXp)
38             binding.badge.isVisible = row.isInPromotionZone
39         }
40     }
41 }

```

写下你的评审。按优先级排序，每条给出：**影响** / **触发条件** / **具体改法**。写完再翻正式书第六章对答案。

0.12 注册了监听器却没有解注册

查错 #12 | 类别: 资源泄漏

场景

单词复习模块有离线队列: 用户在地铁里完成练习, 网络恢复时自动同步。WordReviewActivity 注册了 ConnectivityManager.NetworkCallback, 一旦可用网络出现就触发 sync。代码能工作, 但注册和释放不配对, 长会话后同一个同步动作可能被触发多次, Activity 也可能被系统 callback 持有。

```

1 class WordReviewActivity : AppCompatActivity() {
2     private lateinit var binding: ActivityWordReviewBinding
3     private val syncRepository: WordReviewRepository by lazy {
4         WordReviewRepository.getInstance(this) }
5     private lateinit var connectivityManager: ConnectivityManager
6
7     private val networkCallback = object : ConnectivityManager.NetworkCallback()
8     {
9         override fun onAvailable(network: Network) {
10             lifecycleScope.launch {
11                 syncRepository.syncOfflineQueue()
12                 binding.syncStatus.text = getString(R.string.synced)
13             }
14         }
15     }
16
17     override fun onCreate(savedInstanceState: Bundle?) {
18         super.onCreate(savedInstanceState)
19         binding = ActivityWordReviewBinding.inflate(layoutInflater)
20         setContentView(binding.root)
21         connectivityManager = getSystemService(ConnectivityManager::class.java)
22         connectivityManager.registerDefaultNetworkCallback(networkCallback)
23     }
24
25     override fun onResume() {
26         super.onResume()
27         binding.reviewCard.startListening()
28     }
29
30     override fun onDestroy() {
31         binding.reviewCard.stopListening()
32         super.onDestroy()
33     }
34 }

```

写下你的评审。按优先级排序，每条给出：**影响** / **触发条件** / **具体改法**。写完再翻正式书第六章对答案。

Part III

查错演练：Compose 与 Kotlin 惯用性

0.13 把 MutableStateFlow 暴露出去，并用 SharedFlow 存持久状态

查错 #13 | 类别: **状态暴露**

场景

商店页 ShopViewModel 展示宝石余额、道具卡、订阅入口和购买结果 toast。页面从 SDUI 配置拿卡片，用户旋屏或从支付页返回时应立即看到当前状态。下面代码把可写 StateFlow 直接公开，又把持久 UI 状态和一次性导航事件混在 Flow 里，两个方向都会出错。

```

1  class ShopViewModel(
2      private val shopRepository: ShopRepository,
3      private val purchaseTracker: PurchaseTracker
4  ) : ViewModel() {
5      val uiState = MutableStateFlow(ShopState())
6      val purchaseEvents = MutableStateFlow<PurchaseEvent?>(null)
7      val cards = MutableSharedFlow<List<ShopCard>>()
8
9      fun loadShop() {
10         viewModelScope.launch {
11             uiState.value = uiState.value.copy(loading = true)
12             val response = shopRepository.loadShopCards()
13             cards.emit(response.cards)
14             uiState.value = ShopState(
15                 loading = false,
16                 gemBalance = response.gemBalance,
17                 subscriptionVisible = response.showSubscription
18             )
19         }
20     }
21
22     fun buy(card: ShopCard) {
23         viewModelScope.launch {
24             val receipt = shopRepository.purchase(card.id)
25             purchaseTracker.trackSuccess(card.id)
26             purchaseEvents.value = PurchaseEvent.NavigateToReceipt(receipt.id)
27             uiState.value = uiState.value.copy(gemBalance = receipt.
28 newGemBalance)
29         }
30     }

```

写下你的评审。按优先级排序，每条给出：影响 / 触发条件 / 具体改法。写完再翻正式书第七章对答案。

0.14 LaunchedEffect 的 key 用错，副作用写在 composable 函数体里

查错 #14 | 类别: **Compose 副作用**

场景

课程详情页迁到 Compose 后，需要进入页面时上报曝光，并在 lessonId 变化时重新拉取内容。产品允许用户在同一个容器里切换不同课程；如果 effect key 写错，就会出现两种相反问题：重组时重复请求，或切课后还显示旧课程。

```

1  @Composable
2  fun LessonDetailScreen(
3      lessonId: String,
4      viewModel: LessonDetailViewModel,
5      onStartLesson: (String) -> Unit
6  ) {
7      val state by viewModel.state.collectAsState()
8
9      viewModel.loadLesson(lessonId)
10     viewModel.trackLessonImpression(lessonId)
11
12     LaunchedEffect(Unit) {
13         viewModel.refreshRecommendations(lessonId)
14     }
15
16     Column(modifier = Modifier.fillMaxSize()) {
17         LessonHeader(
18             title = state.title,
19             subtitle = state.subtitle,
20             locked = state.locked
21         )
22         RecommendationCarousel(items = state.recommendations)
23         Button(
24             enabled = !state.locked,
25             onClick = { onStartLesson(lessonId) }
26         ) {
27             Text(text = stringResource(R.string.start_lesson))
28         }
29     }
30 }

```

写下你的评审。按优先级排序，每条给出：影响 / 触发条件 / 具体改法。写完再翻正式书第七章对答案。

0.15 collectAsState 而非 collectAsStateWithLifecycle，以及不稳定参数引发的过度重组

查错 #15 | 类别：重组性能

场景

主界面学习路径 PathScreen 是长列表：顶部有连胜、宝石计数，下面是几十个 lesson node。后台同步 XP、宝石、课程解锁时，上游 Flow 频繁变化。Compose 代码如果不按生命周期收集，不给 LazyColumn key、把不稳定列表和新建 lambda 一路传下去，就会造成后台仍工作、前台整屏重组和滚动位置跳动。

```

1  @Composable
2  fun PathScreen(
3      viewModel: PathViewModel,
4      navigator: LessonNavigator
5  ) {
6      val state by viewModel.uiState.collectAsState()
7      val lessons = state.lessons.toMutableList()
8
9      Column(modifier = Modifier.fillMaxSize()) {
10         TopStatsBar(
11             streak = state.streakDays,
12             gems = state.gemBalance,
13             onShopClick = { navigator.openShop() }
14         )
15
16         LazyColumn(modifier = Modifier.fillMaxSize()) {
17             items(lessons) { lesson ->
18                 LessonNode(
19                     lesson = lesson,
20                     isCurrent = lesson.id == state.currentLessonId,
21                     onClick = { navigator.openLesson(lesson.id) }
22                 )
23             }
24         }
25     }
26 }
27
28 @Composable
29 fun LessonNode(lesson: PathLesson, isCurrent: Boolean, onClick: () -> Unit) {
30     Row(modifier = Modifier.fillMaxWidth().clickable(onClick = onClick)) {
31         Text(text = lesson.title)
32         if (isCurrent) Text(text = stringResource(R.string.current_lesson))
33     }

```

34 }

写下你的评审。按优先级排序，每条给出：**影响 / 触发条件 / 具体改法**。写完再翻正式书第七章对答案。

0.16 !! 强制解包、Java 平台类型与未初始化的 lateinit

查错 #16 | 类别：**空安全**

场景

课程 JSON 从后端下发，LessonRepository 解析后交给 UI 渲染。服务端偶尔省略 subtitle，某个 Java SDK 返回平台类型，adapter 又在快速回调路径上提前访问。Duolingo 官方迁移 Kotlin 时提到历史 NPE crash 是重要背景之一 [\[官方\]²](https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/)；这类代码正是 Kotlin 项目里最该被评审拦下的 NPE 入口。

```

1  class LessonRepository(
2      private val api: LessonApi,
3      private val jsonParser: JavaLessonParser
4  ) {
5      suspend fun loadLesson(id: String): LessonUiModel {
6          val response = api.getLesson(id)
7          val body = response.body()!!
8          val dto = body.data!!
9          val metadata = jsonParser.parse(dto.metadataJson)
10
11         return LessonUiModel(
12             id = dto.id!!,
13             title = dto.title!!,
14             subtitle = metadata.subtitle.uppercase(),
15             unitName = dto.unit!!.name!!,
16             firstExercisePrompt = dto.exercises!![0].prompt!!,
17             authorNameLength = dto.author!!.name!!.length
18         )
19     }
20 }
21
22 class LessonListFragment : Fragment(R.layout.fragment_lesson_list) {
23     private lateinit var adapter: LessonAdapter
24 }

```

²<https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/>

```

25     fun onLessonsLoaded(lessons: List<LessonUiModel>) {
26         adapter.submitList(lessons)
27     }
28
29     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
30         adapter = LessonAdapter()
31         requireView().findViewById<RecyclerView>(R.id.lessons).adapter = adapter
32     }
33 }

```

写下你的评审。按优先级排序，每条给出：影响 / 触发条件 / 具体改法。写完再翻正式书第七章对答案。

0.17 一整段 Java 味的 Kotlin

查错 #17 | 类别：惯用性综合

场景

成就动画页有一段从 Java 机械转换来的 helper：根据用户状态决定 banner 文案、动画类型和下一步动作。它能编译，也能跑，但停在官方迁移文章所说的“自动转换、修编译错误”阶段，还没有进入惯用化重构 [官方]³。查错题里遇到这种代码，不要只说“风格不好”，要把风格和编译期安全、空安全、可维护性连起来。

```

1  class AchievementFormatter private constructor() {
2      companion object {
3          const val STATE_LOCKED = 0
4          const val STATE_UNLOCKED = 1
5          const val STATE_CLAIMED = 2
6
7          @Volatile private var instance: AchievementFormatter? = null
8
9          fun getInstance(): AchievementFormatter {
10             if (instance == null) {
11                 synchronized(AchievementFormatter::class.java) {
12                     if (instance == null) {
13                         instance = AchievementFormatter()
14                     }
15                 }
16             }
17         }
18     }
19 }

```

³<https://blog.duolingo.com/migrating-duolingos-android-app-to-100-kotlin/>

```

17         return instance!!
18     }
19 }
20
21 fun format(user: User?, achievements: List<Achievement>): ArrayList<String>
22 {
23     val result = ArrayList<String>()
24     if (user != null) {
25         result.add(String.format("%s has %d gems", user.getName(), user.
26 getGems()))
27     }
28     if (achievements.size > 0) {
29         for (i in 0 until achievements.size) {
30             val achievement = achievements[i]
31             if (achievement.getState() == STATE_UNLOCKED) {
32                 result.add(String.format("Unlocked: %s", achievement.
33 getTitle()))
34             } else if (achievement.getState() == STATE_CLAIMED) {
35                 result.add(String.format("Claimed: %s", achievement.getTitle
36 ()))
37             } else if (achievement.getState() == STATE_LOCKED) {
38                 result.add(String.format("Locked: %s", achievement.getTitle
39 ()))
40             }
41         }
42     }
43     return result
44 }
45
46 fun animationFor(event: Any): String {
47     if (event is StreakEvent) {
48         return "streak_" + event.days
49     } else if (event is BadgeEvent) {
50         return "badge_" + event.badgeId
51     } else if (event is LeagueEvent) {
52         return "league_" + event.rank
53     }
54     return "none"
55 }
56 }

```

写下你的评审。按优先级排序，每条给出：影响 / 触发条件 / 具体改法。写完再翻正式书第七章对答案。

Part IV

设计演练：核心设计题

0.18 离线课程下载与进度同步

设计题 #1 | 难度: **Hard**

[面经报告] 多份社区面经报告过离线课程、进度同步、学习 session 同步类题目; [官方] Duolingo 官方 frontend prediction 文章也直接讨论了课程进度在离线时先本地递增、恢复网络后同步的场景⁴。因此这题既像真实题, 也高度贴合官方关注点。

0.18.1 题面与常见变体

设计一个 Android 客户端能力: 用户可以下载课程内容, 离线完成 lesson; App 恢复网络后把进度、答题结果和 XP 同步到服务端。常见变体有三种:

- “如何支持离线学习, 并保证进度最终同步?”
- “课程包很大, 下载中断后怎么办? 课程内容更新怎么办?”
- “同一个账号在两台设备离线完成同一课, 回来后如何合并?”

限时 10 分钟口述 (建议录音)。先问澄清问题, 再画模块图, 最后讲存储、离线、失败与回滚。写完再翻正式书第九章对答案。

0.19 连胜 (Streak) 客户端

设计题 #2 | 难度: **Medium**

[面经报告] 社区面经报告过 Streak 客户端设计; [高置信推断] Streak 是 Duolingo 核心产品机制, Android 端必然涉及本地展示、提醒、离线预测与服务端校准。

0.19.1 题面与常见变体

设计 Duolingo Android 的 streak 客户端: 用户每天完成学习后连胜加一; 支持 Streak Freeze; 支持本地提醒; 断网和跨时区时仍给出合理体验。变体包括:

- “今天” 的边界按什么算?
- 离线完成课程后 streak 是否立刻加一?
- 如何调度 “快失去连胜了” 的提醒?

限时 10 分钟口述 (建议录音)。先问澄清问题, 再画模块图, 最后讲存储、离线、失败与回滚。写完再翻正式书第九章对答案。

⁴<https://blog.duolingo.com/frontend-prediction/>

0.20 排行榜客户端与 XP 预测

设计题 #3 | 难度: **Hard**

[高置信推断] 这题未被官方确认为面试原题, 但 **[官方]** Duolingo 官方 frontend prediction 文章详细描述了排行榜 XP 预测、session end card、leaderboard tab 以及三种回滚策略, 因此它是高概率、高价值设计题。

0.20.1 题面与常见变体

设计 Android 排行榜客户端: 用户完成 lesson 后获得 XP, session end card 立即展示 XP 与排名变化; leaderboard tab 展示同组用户排名; 弱网下也要体验顺滑。变体包括:

- “如何在后端计算 XP 较慢时快速展示结果?”
- “预测 XP 和服务端 XP 不一致怎么办?”
- “排行榜晋级能不能也预测?”

限时 10 分钟口述 (建议录音)。先问澄清问题, 再画模块图, 最后讲存储、离线、失败与回滚。写完再翻正式书第九章对答案。

0.21 客户端 A/B 实验 SDK

设计题 #4 | 难度: **Hard**

[高置信推断] 这题没有官方题面, 但 **[官方]** Duolingo 相关 JD 强调跨 iOS、Android、backend、web 持续 A/B tests、reactive Android apps、clean APIs、unit tests; **[官方]** 公司公开文化也高度重视实验。因此客户端实验 SDK 是高置信设计方向。

0.21.1 题面与常见变体

设计 Android A/B 实验 SDK: 业务方可以读取实验 variant, Compose UI 根据 variant 渲染; assignment 稳定、可离线; 曝光准确上报; 实验定义变更后能失效。变体包括:

- “客户端自己哈希分桶, 还是服务端下发 assignment?”
- “如何避免冷启动先显示 control 又闪成 treatment?”
- “曝光什么时候上报, 如何离线去重?”

限时 10 分钟口述 (建议录音)。先问澄清问题, 再画模块图, 最后讲存储、离线、失败与回滚。写完再翻正式书第九章对答案。

Part V

设计演练：进阶设计题

0.22 Server-Driven UI 客户端解析与渲染框架

设计题 #5 | 难度: **Hard**

[官方] Duolingo 已公开介绍其 Server-Driven UI: 响应包含 `screens`、`stylesheet` 与 `data_model`, 并已用于 Shop 与 Purchase Flow。⁵ [高置信推断] 因为这是官方上线且公开撰文的移动端架构系统, 本题是本书里“推断题中概率最高的一道”。

0.22.1 题面与常见变体

- 设计一个 Android 客户端 SDUI 框架, 可以解析服务端下发的页面描述并渲染成原生 UI。
- 如果服务端想免发版改 Shop 或购买流程, 客户端要提供哪些能力集、缓存和版本兼容策略?
- 旧客户端不认识新组件时怎么办? 如果响应有条件展示、点击动作和样式系统, 又该如何测试?

限时 10 分钟口述 (建议录音)。先问澄清问题, 再画模块图, 最后讲存储、离线、失败与回滚。写完再翻正式书第十章对答案。

0.23 图片与音频资产的缓存系统

设计题 #6 | 难度: **Medium**

[高置信推断] Duolingo 是重资产 App: 每个词的发音音频、大量插画、角色动画和课程图片都会影响首屏、课程启动与离线体验, 因此资产缓存是高概率 Android 设计题。

0.23.1 题面与常见变体

- 设计一个图片与音频缓存系统, 支持课程中高频播放单词发音。
- 用户进入一节课前如何预取本节音频, 同时不拖慢前台交互?
- 课程内容更新后, 如何避免播放旧音频或错口音音频?

限时 10 分钟口述 (建议录音)。先问澄清问题, 再画模块图, 最后讲存储、离线、失败与回滚。写完再翻正式书第十章对答案。

⁵<https://blog.duolingo.com/server-driven-ui/>

0.24 推送通知的客户端接收、调度与落地

设计题 #7 | 难度: **Medium**

[面经报告] 多个面经汇总提到通知、提醒与 Android 生命周期问题; [高置信推断] 对 Duolingo 这种依赖每日学习习惯和连胜提醒的产品, 推送通知设计非常自然。

0.24.1 题面与常见变体

- 设计 Duolingo Android 的学习提醒、连胜挽救和营销通知系统。
- FCM 消息到达后, 客户端如何决定展示、去重、落地到具体课程?
- Android 13 通知权限、Android 12 精确闹钟和 Doze 会如何影响设计?

限时 10 分钟口述 (建议录音)。先问澄清问题, 再画模块图, 最后讲存储、离线、失败与回滚。写完再翻正式书第十章对答案。

0.25 间隔重复练习的客户端存储与调度

设计题 #8 | 难度: **Medium**

[官方] / [面经报告] Birdbrain 是 Duolingo 自研个性化选题系统, 在服务端计算 proficiency 并决定给用户出哪些题; [高置信推断] 因此客户端设计的核心不是本地跑 ML, 而是存储、展示、同步和离线降级。

0.25.1 题面与常见变体

- 设计一个“今天该复习哪些词”的 Android 客户端模块。
- 离线时如何继续练习, 联网后又如何接受服务端 schedule?
- 如果客户端本地算法和服务端 Birdbrain 给出的复习计划不同, 谁说了算?

限时 10 分钟口述 (建议录音)。先问澄清问题, 再画模块图, 最后讲存储、离线、失败与回滚。写完再翻正式书第十章对答案。

0.26 把一个 DuoState 式全局状态重构成 MVVM

设计题 #9 | 难度: **Hard**

[官方] Google Android Developers 博客公开描述 Duolingo Android Reboot: 巨型全局可变状态 DuoState 被 XP、streak、lessons 等共享访问, ANR 每月恶化 5-10%, 某次发布崩溃率增加

10%、慢帧 25%、入门设备课程启动慢 70%；团队停更功能 8 周，引入 MVVM、Dagger + Hilt、模块化、Repository pattern 和多线程，最终 ANR 降 41%、帧率不达标时间降 28%、滚动速度提升 40%。⁶ 这题官方基础最扎实，也是最能考察资深度的重构类设计题。

0.26.1 题面与常见变体

- 给你一个被 XP、连胜、课程进度、宝石共享读写的巨型可变状态对象，怎么拆？
- 如何从全局单例迁移到 MVVM + Repository + Hilt，同时不影响发版？
- PM 不愿停 8 周做重构，你如何用数据说服？

限时 10 分钟口述（建议录音）。先问澄清问题，再画模块图，最后讲存储、离线、失败与回滚。写完再翻正式书第十章对答案。

⁶<https://android-developers.googleblog.com/2021/08/android-app-excellence-duolingo.html>